

The background image shows a person's hands typing on a laptop. Overlaid on this are several semi-transparent blue and white data visualization elements, including bar charts, line graphs, pie charts, and circular progress indicators. The overall aesthetic is futuristic and data-driven.

NORMALIZATION

WEEK -13

Relational Model

Database development has the following steps.

- Gather user/business requirements.
- Develop the conceptual E-R Model based on the user/business requirements.
- Convert the E-R Model to a set of relations in the (logical) relational model
- **Normalize the relations to remove any anomalies.**
- Implement the database by creating a table for each normalized relation in a relational database management system.

What is Normalization?

Normalization is a *process* in which we systematically examine relations for *anomalies* and, when detected, remove those anomalies by splitting up the relation into two new, related, relations.

Normalization is an important part of the database development process: Often during normalization, the database designers get their first real look into how the data are going to interact in the database.

What is Normalization?

Finding problems with the database structure at this stage is strongly preferred to finding problems further along in the development process because at this point it is fairly easy to cycle back to the conceptual model (Entity Relationship model) and make changes.

Normalization can also be thought of as a trade-off between data redundancy and performance. Normalizing a relation reduces data redundancy but introduces the need for joins when all of the data is required by an application such as a report query.

Normalization



1NF

First Normal Form

- Ensures that each column contains only atomic values



2NF

Second Normal Form

- Eliminates partial dependencies.



3NF

Third Normal Form

- Eliminates transitive dependencies.



BCNF

Boyce-Codd Normal Form

- Strict version of 3NF that addresses additional anomalies.



4NF

Fourth Normal Form

- Deals with multi-valued dependencies



5NF

Fifth Normal Form

- Addresses join dependencies

StudentID	StudentName	CourseID	CourseName	InstructorID	InstructorName	InstructorOffice
1	John	CS101, CS102	Computer Science, Data Structures	101, 102	Dr. Smith, Dr. Lee	Room 101, Room 102
2	Alice	CS101	Computer Science	101	Dr. Smith	Room 101
3	Bob	CS103	Databases	103	Dr. Brown	Room 103

Step 1: First Normal Form (1NF)

This step will ensure:

- Presence of indivisible values in each column, i.e., single data
- Unique value in each record
- Absence of repeating groups or arrays

StudentID	StudentName	CourseID	CourseName	InstructorID	InstructorName	InstructorOffice
1	John	CS101	Computer Science	101	Dr. Smith	Room 101
1	John	CS102	Data Structures	102	Dr. Lee	Room 102
2	Alice	CS101	Computer Science	101	Dr. Smith	Room 101
3	Bob	CS103	Databases	103	Dr. Brown	Room 103

Second Normal Form (2NF)

To achieve 2NF in DBMS, the table must lack partial dependencies. This means that the table data must depend on the primary key.

In the current example, StudentID and CourseID are the primary keys.

The columns like InstructorName and InstructorOffice vary according to CourseID and not on other values

Student Course Table:

StudentID	StudentName	CourseID	CourseName
1	John	CS101	Computer Science
1	John	CS102	Data Structures
2	Alice	CS101	Computer Science
3	Bob	CS103	Databases

Course Instructor Table:

CourseID	InstructorID	InstructorName	InstructorOffice
CS101	101	Dr. Smith	Room 101
CS102	102	Dr. Lee	Room 102
CS103	103	Dr. Brown	Room 103

Third Normal Form (3NF)

In 3NF, the table must:

Be in 2NF.

Remove transitive dependencies, where non-key attributes depend on other non-key attributes.

Here, InstructorOffice depends on InstructorID.
To resolve this, we create a separate Instructors Table

Course Instructor Table:

CourseID	InstructorID	InstructorName	InstructorOffice
CS101	101	Dr. Smith	Room 101
CS102	102	Dr. Lee	Room 102
CS103	103	Dr. Brown	Room 103

Courses Table:

CourseID	CourseName
CS101	Computer Science
CS102	Data Structures
CS103	Databases

Instructors Table:

InstructorID	InstructorName	InstructorOffice
101	Dr. Smith	Room 101
102	Dr. Lee	Room 102
103	Dr. Brown	Room 103

Course Instructor Table:

CourseID	InstructorID
CS101	101
CS102	102
CS103	103

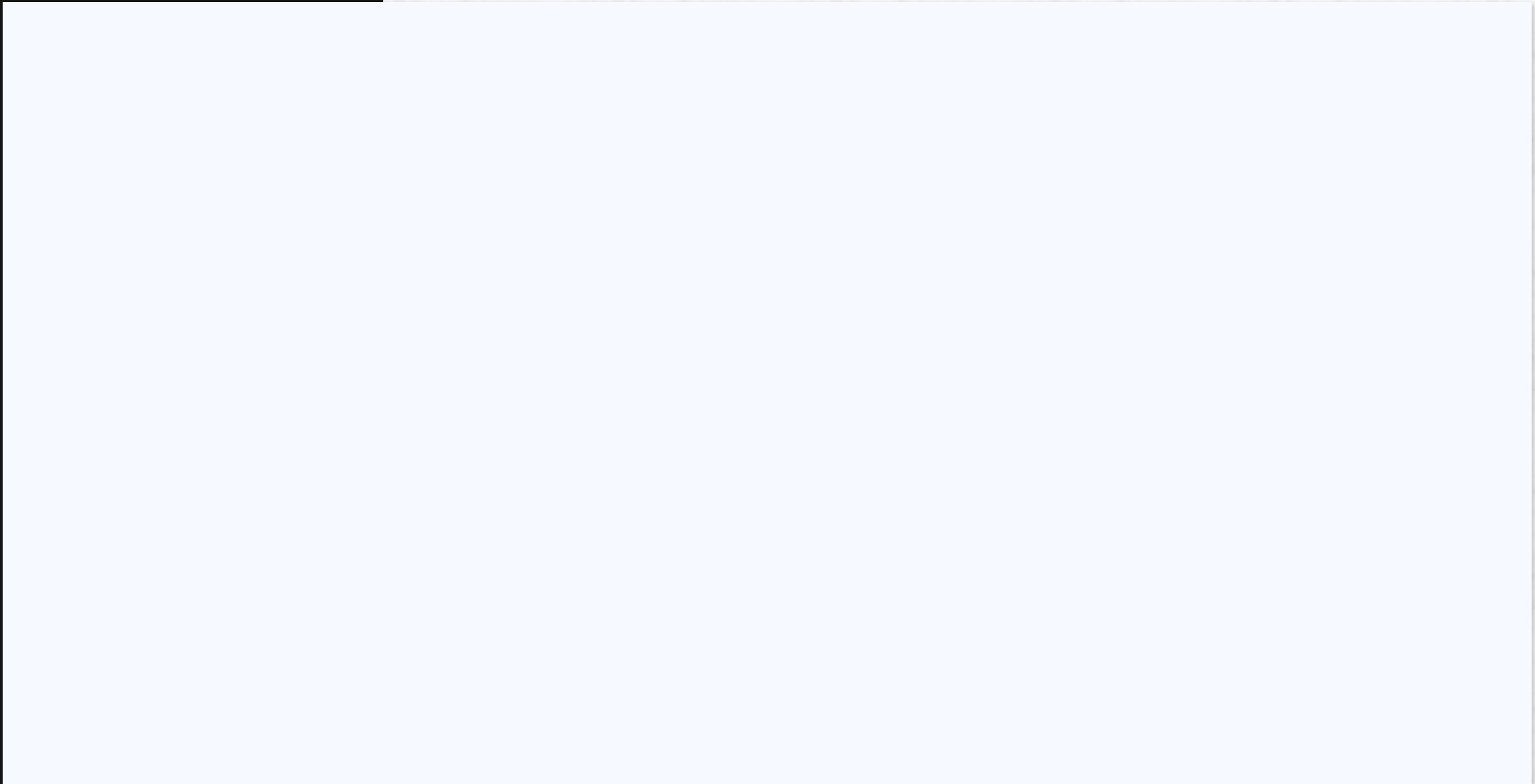
2NF

Rollno	State	City
1	Punjab	Lahore
2	Punjab	Faisalabad
3	Sindh	Karachi
4	Sindh	Larkana

CK=Rollno

FD Rollno $\square$ Province

FD Province $\square$ City



Boyce Codd Normal Form (BCNF)

The first condition for the table to be in Boyce Codd Normal Form is that the table should be in the third normal form.

Secondly, every Right-Hand Side (RHS) attribute of the functional dependencies should depend on the candidate or super key of that particular table.

For example:

You have a functional dependency $X \rightarrow Y$.

In the particular functional dependency, X has to be part of the candidate or super key of the provided table.

Rollno	Name	Voter ID	Age
1	Ali	K101	20
2	Ayesha	K102	21
3	Ikram	K103	20
4	Irum	K104	23

CK-(Rollno, voter ID)

FD Rollno $\square$ Name

FD Rollno $\square$ Voter ID

FD Voter ID $\square$ Age

FD Voter ID $\square$ Rollno

Subject table

	stuid	subject	professor
▶	1	SQL	Prof. Mishra
	2	Java	Prof. Anand
	2	C++	Prof. Kanth
	3	Java	Prof. James
	4	DBMS	Prof. Lokesh

Subject table

The subject table follows these conditions:

1. Each student can enroll in multiple subjects.
2. Multiple professors can teach a particular subject.
3. For each subject, it assigns a professor to the student.

	stuid	subject	professor
▶	1	SQL	Prof. Mishra
	2	Java	Prof. Anand
	2	C++	Prof. Kanth
	3	Java	Prof. James
	4	DBMS	Prof. Lokesh

Subject table

In the table, student_id and subject together form the primary key because by using student_id and subject, you can determine all the table columns.

Another important point is that one professor teaches only one subject, but one subject may have two professors.

This exhibits a dependency between the subject and the professor, i.e., the subject depends on the professor's name.

	stuid	subject	professor
▶	1	SQL	Prof. Mishra
	2	Java	Prof. Anand
	2	C++	Prof. Kanth
	3	Java	Prof. James
	4	DBMS	Prof. Lokesh

1. The table is in 1st Normal form as all the column names are unique, all values are atomic, and all the values stored in a particular column are of the same domain.
2. The table also satisfies the 2nd Normal Form, as there is no Partial Dependency.
3. There is no Transitive Dependency; hence, the table satisfies the 3rd Normal Form.
4. This table follows all the Normal forms except the Boyce Codd Normal Form.

1. stuid and subject form the primary key, which means the subject attribute is a prime attribute.
2. However, there exists yet another dependency - professor $\rightarrow$ subject.
3. BCNF does not follow in the table as a subject is a prime attribute, and the professor is a non-prime attribute.

To transform the table into the BCNF, you will divide the table into two parts. One table will hold stuid, which already exists, and the second table will hold a newly created column profid.

	stuid	profid
►	1	101
	2	102
	2	103
	3	102
	4	104

The second table will have profid, subject, and professor columns, which satisfies the BCNF.

	profid	subject	professor
►	1	SQL	Prof. Mishra
	2	Java	Prof. Anand
	2	C++	Prof. Kanth
	3	Java	Prof. James
	4	DBMS	Prof. Lokesh

Example

A table is in BCNF if each determinant is a candidate key. In other words, every non-trivial functional dependency in the table must be on a candidate key. For example, consider a table that lists information about books and their authors ?

Table: Books

Book ID	Title	Author ID	Author Name	Author Nationality
1	Crime and Punishment	100	Fyodor Dostoevsky	Russian
2	The Great Gatsby	101	F. Scott Fitzgerald	American
3	Pride and Prejudice	102	Jane Austen	British

In this example, the functional dependency between "Author ID" and "Author Name" violates BCNF because it is not on a candidate key. To bring this table to BCNF, we can split it into two tables ?

Table 1: Authors

Author ID	Author Name	Author Nationality
101	Fyodor Dostoevsky	Russian
101	F. Scott Fitzgerald	American
102	Jane Austen	British

Table 2: Books

Book ID	Title	Author ID
1	Crime and Punishment	100
2	The Great Gatsby	101
3	Pride and Prejudice	102

Now, the "Author Name" and "Author Nationality" columns are not transitively dependent on the primary key, and the table is in BCNF.

Fourth Normal Form (4NF)

A multi-valued dependency occurs when a non-primary key column depends on a combination of other non-primary key columns.

Multivalued dependent Mobile and email no relation

Name	Mobile no	Email
Varun	01	E1
Varun	01	E2
Varun	01	E3
Varun	02	E1
Varun	02	E2
Varun	02	E3
Varun	03	E1
Varun	03	E2
Varun	03	E3

4th Normal Form

- A relation will be in 4NF if it is in Boyce Codd normal form
- It has no multi-valued dependency.
- For a dependency $A \twoheadrightarrow B$, if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.

What is multi-valued dependency?

Student_ID	Course	Hobby
21	Computer	Dancing
21	Math	Singing
34	Chemistry	Dancing
74	Biology	Cricket
59	Physics	Hockey

Student_id -> Course

Student_id -> Hobby

4th Normal Form

Student_Course Table

Student_ID	Course
21	Computer
21	Math
34	Chemistry
74	Biology
59	Physics

Student_Hobby Table

Student_ID	Hobby
21	Dancing
21	Singing
34	Dancing
74	Cricket
59	Hockey

First normal form (1NF)

As per the rule of first normal form,

- An attribute (column) of a table cannot hold multiple values.
- It should hold only **atomic** (forming a single unit) values.
- Each record needs to be unique.

Atomic Value

An atomic value is a value that cannot be divided.

Employee Table			
Emp_Id	Name	MobileNo	Dept
E1001	John	3343434	Sales
E1002	Smith	5454546, 5454547	Purchase
E1003	Kim	6565656, 6565657	IT
E1004	Arnold	90876545	Sales
E1005	Moba	8787656	Accounts

First Normal Form(1NF)

Not a Atomic
value

Employee Table			
Emp_Id	Name	MobileNo	Dept
E1001	John	3343434	Sales
E1002	Smith	5454546, 5454547	Purchase
E1003	Kim	6565656, 6565656	IT
E1004	Arnold	90876545	Sales
E1005	Moba	8787656	Accounts

After 1st Normalisation

Employee Table			
Emp_Id	Name	MobileNo	Dept
E1001	John	3343434	Sales
E1002	Smith	5454546	Purchase
E1002	Smith	5454547	Purchase
E1003	Kim	6565656	IT
E1003	Kim	6565657	IT
E1005	Moba	8787656	Accounts

2nd Normal Form Definition

- It should be in the First Normal form.
- All non-key attributes are fully functional dependent on the primary key. In simple words it should not have Partial Dependency.

Emp_id	Qualification	Age
111	MA	38
111	BTEch	38
222	MCA	38
333	MS	40
333	MBA	40

Composite Key: **Emp_ID+Qualification**

Non-Key Attribute

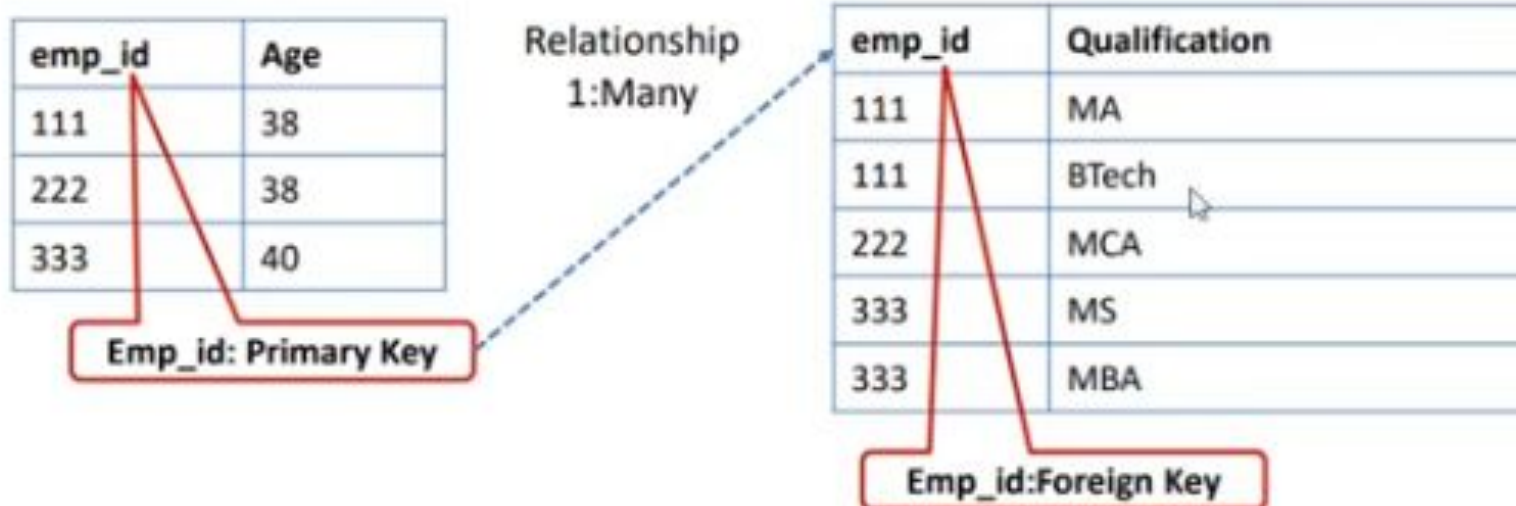
- Composite primary key : **Emp_ID+Qualification**
- Non-key attribute: **Age**

How to find age of employee

Activate

2nd Normal Form Definition

➤ To make the table complies with 2NF we can break it in two tables like this:



1st NF

- It should hold only **atomic** (forming a single unit) values.
- Each record needs to be unique.

2nd NF

- Non-key attribute should depend on single key

3rd Normal Form Definition

- It is in second normal form
- There is no **transitive functional dependency**.
- A transitive functional dependency is when changing a non-key column, might cause any of the other non-key columns to change.

Emp_ID	Name	ZIP	City
E100	Harry	20101	Noida
E101	Stephan	02228	Boston
E102	Lan	60007	Chicago
E103	Katharine	06389	Norwich
E104	John	02228	Boston

Emp_id: Primary Key

Change in **zip** requires change in City

Emp_id -> Zip -> City	City dependent on Zip dependent on Emp_id
Emp_id -> City	City dependent on Emp_id City Indirectly dependent on Emp_id is called Transitive Functional Dependency SOLUTION: Create New Table for Zip and City

Activate V
Go to Settings

3rd Normal Form Solution

Emp Table

Emp_ID	Name	ZIP
E100	Harry	20101
E101	Stephan	02228
E102	Lan	60007
E103	Katharine	06389
E104	John	02228

Emp_id: Primary Key

Zip: Foreign Key

Zip Table

ZIP _i	City
20101	Noida
02228	Boston
60007	Chicago
06389	Norwich

Zip: Primary Key

Boyce-Codd Normal Form (BCNF)

- **Boyce-Codd Normal** Form or BCNF is an extension to the third normal form, and is also known as 3.5 Normal Form.
- In BCNF if every functional dependency $A \rightarrow B$, then A has to be the **Super Key** (is a key that uniquely identifies rows in a table) of that particular table.

student_id	subject	professor
101	Java	P.Java
101	C++	P.Cpp
102	Java	P.Java2
103	C#	P.Chash
104	Java	P.Java

- One student can enroll for multiple subjects.
- For each subject, a professor is assigned to the student.
- And, there can be multiple professors teaching one subject.

student_id + subject(**primary key** , subject as prime attribute)

professor -> subject

subject is a prime attribute, professor is a non-prime attribute, which is not allowed by BCNF.

Boyce-Codd Normal Form (BCNF)

➤ . Student able

student_id	professor_id
101	1
101	2
102	3
103	4
104	1

Professors table^I

professor_id	professor	subject
1	P.Java	Java
2	P.Cpp	C++
3	P.Java2	Java
4	P.Chash	C#

For example, a table that lists customer orders with a primary key of order ID and non-primary key columns for customer ID and order items violates 4NF because order items depend on both order ID and customer ID.

For example, a table that lists orders and their products, with columns for order ID, product ID, and product details, violates 4NF because the product details depend on the combination of order ID and product ID.

Example

Consider the following table of orders and products

Order ID	Product ID	Product Name	Product Description
1	100	Widget	Red Widget
1	200	Widget	Blue Widget
2	100	Widget	Red Widget
2	300	Thing	Green Thing
3	200	Widget	Blue Widget
3	300	Thing	Green Thing

In this table, the product name and description depend on both the order ID and product ID, creating a multi-valued dependency. To bring the table into 4NF, we can split it into three tables ?

Order ID	Product ID
1	100
1	200
2	100
2	300
3	200
3	300

--	--

--

Product ID	Product Name
100	Widget
200	Widget
300	Thing
Product ID	Product Description
100	Red Widget
200	Blue Widget
300	Green Thing